CCNA（200-301 v1.1）　6.0 自動化とプログラマビリティ　#25

REST API・構成管理・JSON

出題範囲 6.5〜6.7 ― REST API の特徴（認証・CRUD・HTTP の動詞・データの形式）、構成管理ツール Ansible と Terraform の見分け方、JSON の読み方

各ページの下に、根拠にしたCiscoの資料やRFCなどを書いた（青い文字を押すと開く）

全体像 6.5 は Describe（説明）、6.6・6.7 は Recognize（見分ける）

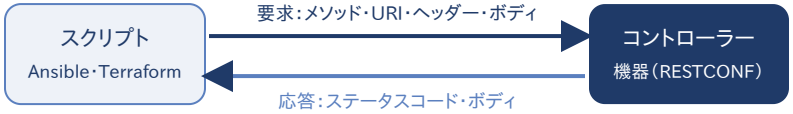
1 REST API のしくみ（6.5）
要求と応答の部品（メソッド・URI・ヘッダー・ボディ）、ステートレス、RESTCONF

2 CRUD と HTTP の動詞（6.5）
GET・POST・PUT・PATCH・DELETE、安全とべき等、ステータスコード、4つの認証

3 構成管理（6.6）
Ansible（エージェントレス・YAML・SSH）と Terraform（HCL・宣言型・ステートファイル）

4 JSON（6.7）
オブジェクト・配列・名前と値・値の型。XML・YAML との見分け

#24 のノースバウンド API の中身



ボディの中身は JSON（または XML）

{"hostname": "SW1",
"vlans": [10, 20]}

（Claude が作成した例）

芯：REST API は **HTTP** の上で「どのリソースに（URI）・何をするか（メソッド）」を送り、**ステータスコード**と **JSON** で答えが返る。

出典：Cisco CCNA Exam v1.1 (200-301) Exam Topics（6.5 Describe characteristics of REST-based APIs (authentication types, CRUD, HTTP verbs, and data encoding)／6.6 Recognize the capabilities of configuration management mechanisms such as Ansible and Terraform／6.7 Recognize components of JSON-encoded data）

第1章

REST API のしくみ

HTTP の要求と応答で、リソースを読み書きする。

要求と応答の部品

要求（クライアント → サーバー）

```
GET /restconf/data/Cisco-IOS-XE-native:native/hostname HTTP/1.1
Host: 10.1.1.1
Accept: application/yang-data+json
Authorization: Basic YWRtaW46…
```

応答（サーバー → クライアント）

```
HTTP/1.1 200 OK
Content-Type: application/yang-data+json

{"Cisco-IOS-XE-native:hostname": "SW1"}
```

RESTCONF での例。Cisco の資料の要求と応答をもとに Claude が組み直した。

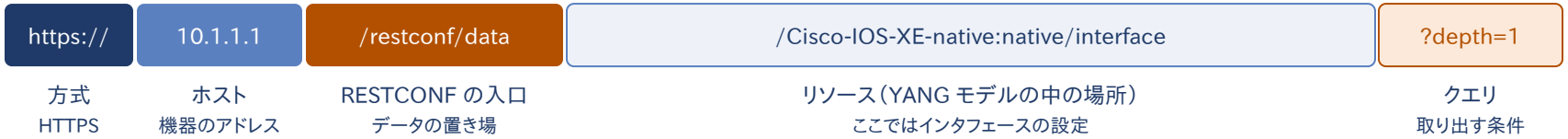
部品	中身
メソッド	何をしたいか（**GET**・POST・PUT・PATCH・DELETE）
URI	どのリソースに（**エンドポイント**）
ヘッダー	付け足す情報。**Accept**＝ほしい形式、**Content-Type**＝ボディの形式、**Authorization**＝認証
ボディ	送る・返るデータ（**JSON**・XML）
ステータスコード	結果（200・201・401・404 など）

ステートレス：サーバーは前の要求を覚えない。だから**要求ごとに**認証の情報（トークンなど）を付ける。Cisco は RESTCONF を「HTTPS を使うステートレスなプロトコル」と説明している。

出典：Cisco IOS XE 17.9 Programmability CG「RESTCONF Protocol」（The HTTPS-based RESTCONF protocol (RFC 8040), is a stateless protocol that uses secure HTTP methods／GET /restconf HTTP/1.1 … Accept: application/yang-data+json … HTTP/1.1 200 OK … Content-Type: application/yang-data+json）／RFC 9110（HTTP Semantics）

URI の読み方と RESTCONF

URI を分けて読む（Cisco の資料の例）



RESTCONF の要点	中身
何か	HTTPS で、YANG で決めたデータ（設定・状態）を CRUD する（RFC 8040）
データの形式	application/yang-data+json か +xml
NETCONF との違い	NETCONF は SSH の上で XML。データの考え方は同じ

IOS XE で有効にする（AAA の設定も要る）

```
Device(config)# restconf
Device(config)# ip http secure-server
```

落とし穴：RESTCONF は **HTTPS** を使う。SSH の設定や SNMP の設定では使えるようにならない。

出典：Cisco IOS XE 17.9 Programmability CG「RESTCONF Protocol」（depth=1 - https://10.85.116.59/restconf/data/Cisco-IOS-XE-native:native/interface/GigabitEthernet?depth=1／provide CREATE, READ, UPDATE, and DELETE (CRUD) operations on a conceptual datastore containing YANG-defined data, which is compatible with a server that implements NETCONF datastores／Application/YANG-Data+XML OR Application/YANG-Data+JSON／Enabling Cisco IOS HTTP Services for RESTCONF: restconf, ip http secure-server／NETCONF and RESTCONF connections must be authenticated using AAA）。URI のアドレスは Claude が変えた

第2章

CRUD・HTTP の動詞・認証

何をするかはメソッドで、結果はステータスコードで、誰かは認証で伝える。

CRUD と HTTP のメソッド

CRUD	メソッド	安全	べき等
Create	POST（PUT でも作れる）	×	×
Read	GET	○	○
Update	PUT（置き換え） PATCH（一部）	×	PUT ○ PATCH 決まりなし
Delete	DELETE	×	○

RESTCONF の表：GET＝Read、PATCH＝Update、PUT＝Create or Replace、POST＝Create、DELETE＝消す。

安全（safe）：サーバーの状態を変えない。**GET**・HEAD・OPTIONS・TRACE。

べき等（idempotent）：同じ要求を何回送っても、1回と同じ結果。**PUT・DELETE** と安全なメソッド。

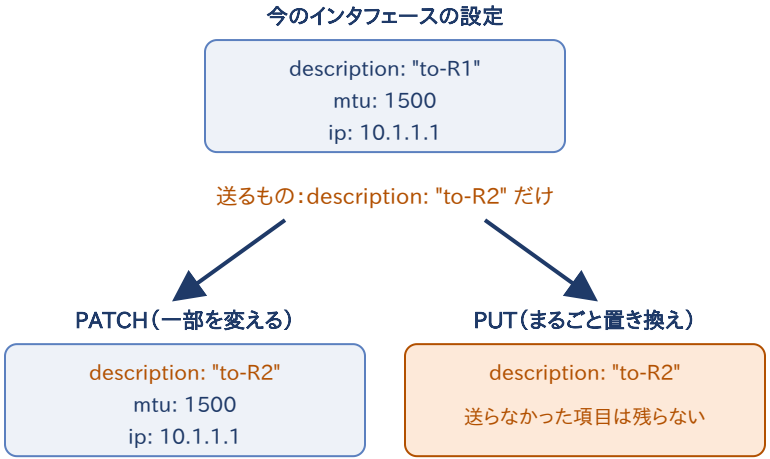
落とし穴：POST はべき等ではない。同じ POST を2回送ると、2つ作られることがある。送り直しで安心なのは PUT・DELETE。

DELETE を2回
消えた状態は同じ（べき等）

POST を2回
2つできることがある

出典：RFC 9110 9.2.1・9.2.2（the GET, HEAD, OPTIONS, and TRACE methods are defined to be safe／PUT, DELETE, and safe request methods are idempotent）／RFC 5789（PATCH）／Cisco IOS XE 17.9 Programmability CG「RESTCONF Protocol」（GET Read／PATCH Update／PUT Create or Replace／POST Create or Operations／DELETE Deletes the targeted resource）

PUT と PATCH の違い



考え方を示した図（Claude が作成）

	PUT	PATCH
すること	送った内容で**作る・まるごと置き換える**	送った変更だけを**当てる**
送らなかった部分	置き換わる	**そのまま**
べき等	○	決まりなし
定義	RFC 9110	RFC 5789

使い分け：「説明だけ変えたい」→ **PATCH**。「この内容にそっくりそろえたい」→ **PUT**。

出典：RFC 9110 9.3.4（The PUT method requests that the state of the target resource be created or replaced with the state defined by the representation enclosed in the request message content）／RFC 5789 2.（The PATCH method requests that a set of changes described in the request entity be applied to the resource）

ステータスコード

1けた目	種類
1xx	情報（続けて処理中）
2xx	**成功**
3xx	リダイレクト（別の場所へ）
4xx	**クライアント**の誤り
5xx	**サーバー**の誤り

落とし穴：4xx は要求を送った**こちら側**の問題、5xx は**サーバー側**の問題。

コード	意味	よくある場面
200 OK	成功	GET で返った
201 Created	成功して**新しくできた**	POST で作った
204 No Content	成功、返す中身なし	DELETE のあと
400 Bad Request	要求の書き方がおかしい	JSON の文法の誤り
401 Unauthorized	正しい**認証の情報がない**	トークンの期限切れ
403 Forbidden	分かったが**断る**	権限が足りない
404 Not Found	リソースが**見つからない**	URI の打ち間違い
500	サーバーの中で思わぬ問題	サーバーの不具合

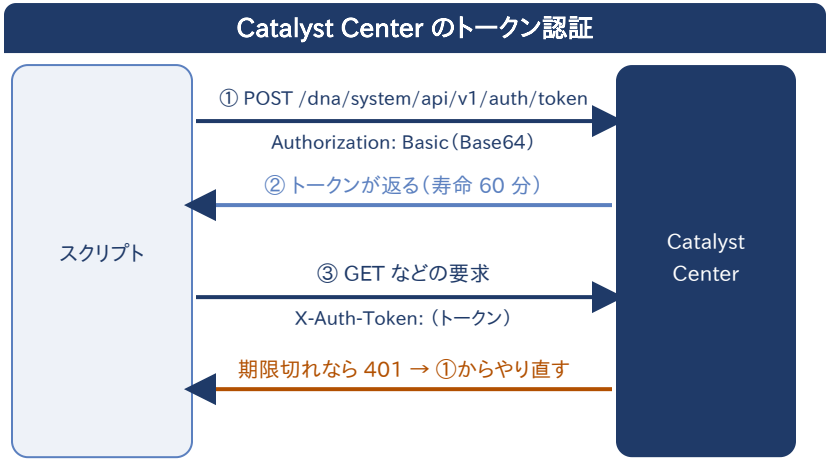
「よくある場面」は Claude が書いた例。

出典：RFC 9110 15.（1xx (Informational) … 2xx (Successful) … 3xx (Redirection) … 4xx (Client Error) … 5xx (Server Error)）・15.3.1 200・15.3.2 201（resulted in one or more new resources being created）・15.3.5 204・15.5.1 400・15.5.2 401（lacks valid authentication credentials）・15.5.4 403（understood the request but refuses to fulfill it）・15.5.5 404・15.6.1 500

REST API の認証

方式	送るもの	注意
Basic	「ユーザー名:パスワード」を **Base64**	暗号化ではない。**HTTPS** と使う
トークン	先にもらった**トークン**（Bearer・X-Auth-Token）	持つ人なら誰でも使える。期限あり
API キー	前もって発行した**鍵**	パスワードと同じに扱う。ソースに書かない
OAuth 2.0	持ち主が**許可した範囲**の権限	アプリにパスワードを渡さない

Meraki のダッシュボード API は、API キー（管理者の権限で動く）と OAuth（アプリの連携）の2つ。どちらも Authorization: Bearer で送る。



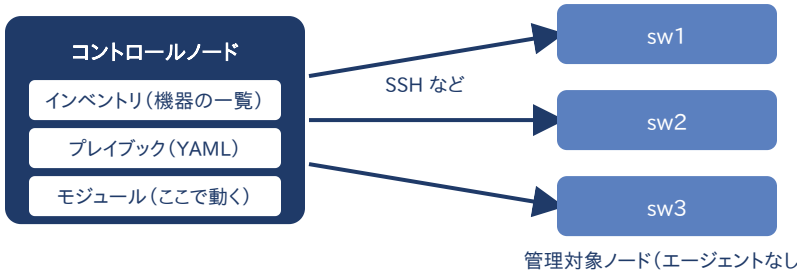
出典：RFC 7617（credentials as user-id/password pairs, encoded using Base64／not considered to be a secure method … unless used in conjunction with … TLS）／RFC 6750（any party in possession of a bearer token … can use it）／RFC 6749（enables a third-party application to obtain limited access to an HTTP service）／Cisco DevNet「Catalyst Center API: Authentication」（POST /dna/system/api/v1/auth/token／X-Auth-Token／The lifetime of a token is 60 minutes … returns a 401 UNAUTHORIZED）／Meraki Dashboard API v1「Authorization」（Admin-scoped access - Uses API keys／OAuth／Bearer Auth using the standard Authorization header）

第3章

構成管理（Ansible と Terraform）

設定や基盤をコードで書き、そのとおりの状態にそろえる。

Ansible




```
- name: NTP をそろえる
  hosts: switches
  gather_facts: false
  tasks:
    - name: NTP サーバーを入れる
      ansible.netcommon.cli_config:
        config: ntp server 10.1.1.10
```

プレイブックの例（Claude が作成）。hosts のグループはインベントリで決める。

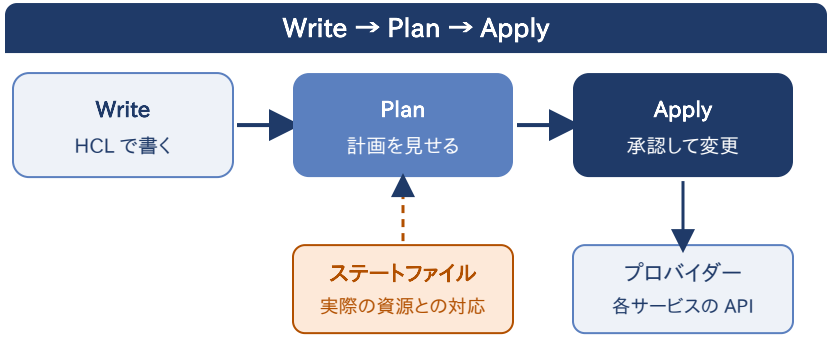
部品	中身
コントロールノード	Ansible を入れてコマンドを実行する PC・サーバー
管理対象ノード	管理される機器。Ansible は**入れない**
インベントリ	機器の一覧と**グループ**、IP アドレスなど
プレイブック	YAML。どの機器に、どのタスクを順に行うか
モジュール	タスクで実際の仕事をする部品

> **特徴：エージェントレス**・YAML で読みやすい・**SSH** で入る・**べき等**（すでに望む状態なら何も変えない）。Python で書かれている。

ネットワーク機器のモジュールは機器の上でなくコントロールノードで動き、CLI over SSH（最も多い）・XML over SSH・API over HTTPS で話す。

出典：Ansible「Introduction to Ansible」（Agent-less architecture／straightforward YAML syntax／using SSH with existing OS credentials／When the system is in the state your playbook describes, Ansible does not change anything）／同「Basic Concepts」（Control node／Managed nodes … Ansible is not normally installed on managed nodes／Inventory … assigning groups／Playbooks are written in YAML）／同「How Network Automation is Different」（network modules do not run on the managed nodes／XML over SSH, CLI over SSH, API over HTTPS … The most common protocol is CLI over SSH／Ansible is written and executed in Python）

Terraform



plan だけでは何も変わらない

```
provider "aws" {
  region = var.aws_region
}
resource "aws_vpc" "main" {
  cidr_block = var.base_cidr_block
}
```

HashiCorp の資料の HCL の例から抜き出した。

言葉	中身
IaC	基盤を人が読める**構成ファイル**で定義し、バージョン管理・使い回しをする
HCL・宣言型	**できあがりの状態**だけを書く。順番は Terraform が依存関係から決める
プロバイダー	クラウドなどの API とやり取りするプラグイン
plan と apply	plan＝作る・変える・消すの**計画**。apply＝承認して**実行**
ステートファイル	作った資源の記録。環境の「**正しい情報のもと**」

出典：HashiCorp「What is Terraform?」(infrastructure as code tool … human-readable configuration files that you can version, reuse, and share／Write … Plan … Apply: On approval／state file, which acts as a source of truth／declarative, meaning that they describe the end state)／同「Providers」(plugins called providers to interact with cloud providers … and other APIs)／同「terraform plan」(The plan command alone does not actually carry out the proposed changes)／同「Terraform Language」(declarative／The ordering of blocks … generally not significant)／同「Configuration Syntax」(HCL)

Ansible と Terraform を見分ける

	Ansible	Terraform
種類	オープンソースの**自動化**ツール（Python）	HashiCorp の **IaC** ツール
書く言葉	**YAML**（プレイブック）	**HCL**（構成ファイル）
書き方	望む状態を書いた**タスクを順に**実行	**宣言型**。できあがりだけ書き、順番はツールが決める
相手への入り方	**エージェントなし**。SSH（ネットワーク機器は CLI over SSH など）	**プロバイダー**が各サービスの API を呼ぶ
状態の記録	持たない	**ステートファイル**
共通点	設定をファイル（コード）にして、同じ状態を何度でも作れる。すでに望む状態なら何も変えない（**べき等**）	

見分けのキーワード：YAML・プレイブック・インベントリ・エージェントレス → **Ansible**。HCL・プロバイダー・plan／apply・ステート → **Terraform**。

落とし穴：「Ansible は機器にエージェントを入れる」は誤り。「Terraform は YAML で書く」も誤り。

出典：Ansible「Introduction to Ansible」・「How Network Automation is Different」／HashiCorp「What is Terraform?」（If Terraform detects that no changes are needed … will report that no actions need to be taken：terraform plan のページ）

第4章

JSON

API の答えを読むための、6つの部品。

JSON の部品を読む

```
{
    "hostname": "SW1",
    "mtu": 1500,
    "enabled": true,
    "location": null,
    "vlans": [10, 20, 30],
    "stp": {
        "mode": "rapid-pvst"
    }
}
```

← 外側の { }=オブジェクト
← 文字列(" ")
← 数値(引用符なし)
← 真偽値(小文字)
← null(値なし)
← 配列 [ ]
← 中のオブジェクト

オブジェクトは 2 個

部品	決まり
オブジェクト	{ } の中に「名前: 値」を , で並べる。順番に意味はない
配列	[ ] の中に値を , で並べる。順番に意味がある
名前(キー)	必ず**文字列**(ダブルクォート)。重ならないようにする
基本の型	文字列・数値・真偽値・null の**4つ**
組み立てる型	オブジェクト・配列の**2つ**
空白・改行	意味はない(読みやすくするため)

Claude が作成した例。

出典:RFC 8259(JSON can represent four primitive types (strings, numbers, booleans, and null) and two structured types (objects and arrays)／An object is an unordered collection of zero or more name/value pairs, where a name is a string／An array is an ordered sequence of zero or more values／Insignificant whitespace is allowed／The names within an object SHOULD be unique)

よくある JSON の誤りと、XML・YAML

誤り	正しくは
'hostname'	"hostname"（ダブルクォート）
True・NULL	true・null（小文字だけ）
[10, 20, 30,]	最後の値のあとに , を付けない
"mtu" 1500	名前と値の間に :
"mtu": 01500	数値の頭に余計な 0 を付けない

落とし穴："10" は文字列、10 は数値。"true" も文字列。

同じ「VLAN 10・名前 SALES」を3つの形式で（Claude が作成）

形式	書き方
JSON	`{"vlan": {"id": 10, "name": "SALES"}}`
XML	`<vlan><id>10</id><name>SALES</name></vlan>`

YAML	`vlan:`
	`  id: 10`
	`  name: SALES`

見分け方：{ } と [ ] → JSON、<タグ> → XML、字下げと「名前: 値」→ YAML（Ansible のプレイブック）。RESTCONF は JSON か XML。

出典：RFC 8259（A string begins and ends with quotation marks／The literal names MUST be lowercase／array = begin-array [value *(value-separator value)] end-array／name-separator = ws %x3A ws ; : colon／Leading zeros are not allowed）／Ansible「YAML Syntax」（A dictionary is represented in a simple key: value form）／IOS XE 17.9 Programmability CG「RESTCONF Protocol」（Application/YANG-Data+XML OR Application/YANG-Data+JSON）

よくあるひっかけ

よくある思い込み	正しくは
POST はべき等	べき等なのは **PUT・DELETE** と GET など。POST は違う
一部だけ変えるのは PUT	一部は **PATCH**。PUT はまるごと置き換え
401 は権限が足りないとき	401＝**認証の情報がない**。権限が足りないのは **403**
5xx は要求の書き方の誤り	書き方の誤りは **4xx**（400）。5xx はサーバー側
Basic 認証はパスワードを暗号化する	**Base64** はただの変換。HTTPS と使う
Ansible は機器にエージェントを入れる	**エージェントレス**。SSH で入る
Terraform は YAML のプレイブック	**HCL** の構成ファイル。プレイブックは Ansible
JSON の真偽値は True	小文字の **true**。"true" は文字列
JSON の配列は { }	配列は [ ]、オブジェクトが { }

出典：このPDFの各ページの資料

まとめ

REST API のしくみ
HTTP の上でメソッド+URI+ヘッダー+ボディ。ステートレス。RESTCONF は HTTPS で YANG のデータを JSON／XML で CRUD

CRUD と認証
POST／GET／PUT・PATCH／DELETE。安全=GET、べき等=PUT・DELETE。2xx 成功・4xx こちら・5xx サーバー。Basic・トークン・API キー・OAuth

Ansible と Terraform
Ansible=エージェントレス・YAML・SSH・インベントリ。Terraform=HCL・宣言型・プロバイダー・plan／apply・ステート

JSON
{ } オブジェクト、[ ] 配列、"名前": 値。文字列・数値・true／false・null

数字・名前	中身
Catalyst Center のトークン	X-Auth-Token・60 分
トークンを取る	POST …/auth/token
RESTCONF の形式	yang-data+json／+xml
新しくできた	201 Created
認証がない	401

次の分野：#01 ネットワーク機器と構成で、ルーター・スイッチ・ファイアウォール・AP などの役割と、2 層・3 層のつなぎ方を読む。

出典：このPDFの各ページの資料